

WebApi Handson Solution

5. WebApi Hands-on Solution

Objectives Addressed

- * CORS enablement for Web API.
- * Bearer and JWT token authentication.
- * Role-based Authorization (`[Authorize(Roles="...")]`).

Solution Implementation

1. Enable CORS and Authentication Setup

In `Program.cs` (or `Startup.cs`), add CORS and JWT Bearer settings:

```
```csharp
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddCors();

string securityKey = "mysuperdupersecret";
var symmetricSecurityKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(securityKey));

builder.Services.AddAuthentication(x =>
{
 x.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
 x.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
 x.DefaultSignInScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(JwtBearerDefaults.AuthenticationScheme, x =>
{
 x.TokenValidationParameters = new TokenValidationParameters
```

# WebApi Handson Solution

```
{
 ValidateIssuer = true,
 ValidateAudience = true,
 ValidateLifetime = true,
 ValidateIssuerSigningKey = true,
 ValidIssuer = "mySystem",
 ValidAudience = "myUsers",
 IssuerSigningKey = symmetricSecurityKey
};
});

builder.Services.AddControllers();

var app = builder.Build();

app.UseCors(options => options.AllowAnyOrigin());
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
app.Run();
````
```

2. AuthController

This controller generates the JWT Token.

```
``csharp
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.IdentityModel.Tokens;
using System;
using System.Collections.Generic;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
```

WebApi Handson Solution

```
namespace WebApi_Handson.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class AuthController : ControllerBase
    {
        [AllowAnonymous]
        [HttpGet]
        public IActionResult Get()
        {
            var token = GenerateJSONWebToken(1, "Admin");
            return Ok(new { token = token });
        }

        private string GenerateJSONWebToken(int userId, string userRole)
        {
            var securityKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes("mysuperdupersecret"));
            var credentials = new SigningCredentials(securityKey, SecurityAlgorithms.HmacSha256);

            var claims = new List<Claim>
            {
                new Claim(ClaimTypes.Role, userRole),
                new Claim("UserId", userId.ToString())
            };

            var token = new JwtSecurityToken(
                issuer: "mySystem",
                audience: "myUsers",
                claims: claims,
                expires: DateTime.Now.AddMinutes(2), // Expiration modified to 2 minutes
                signingCredentials: credentials);

            return new JwtSecurityTokenHandler().WriteToken(token);
        }
    }
}
```

WebApi Handson Solution

```
}  
}  
...
```

3. EmployeeController Authorization

Apply the `[Authorize]` attribute to enforce role-based access control.

```
```csharp  
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace WebApi_Handson.Controllers
{
 [Route("api/[controller]")]
 [ApiController]
 // Role requirements: Admin or POC
 [Authorize(Roles = "Admin,POC")]
 public class EmployeeController : ControllerBase
 {
 [HttpGet]
 public IActionResult Get()
 {
 return Ok(new[] { "Secured", "Data" });
 }
 }
}
```
```

Testing Process

1. Make a GET request to `/api/Auth` to receive the token.
2. Pass the token in the `Authorization` header as `Bearer <token>` in Postman when calling `/api/Employee`.
3. Wait 2 minutes and try the API call again; it will return `401 Unauthorized`.